

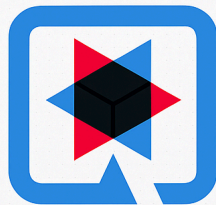
Documentación Prueba Técnica

Moises Pinzon Xiques

Tecnologo: Analisis y Desarrollo de sistemas de la Información(Sena)

Estudiante Ingeniería Informática 6º Semestre(Universidad
Militar Nueva Granada)

GETTING STARTED WITH



QUARKUS

**Build Your First Java
Project Like a Pro**

2026

Documentación del Microservicio: ms-risk

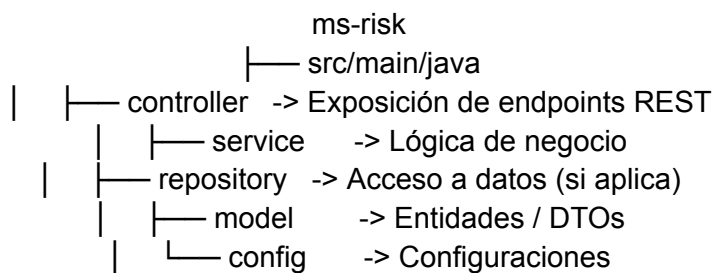
Descripción general

ms-risk es un microservicio desarrollado en **Java con Quarkus** que forma parte de un sistema de crédito. Su objetivo principal es **evaluar el riesgo de un cliente** basado en reglas de negocio definidas.

Este servicio puede ser consumido por otros microservicios dentro de la arquitectura del sistema financiero.

Arquitectura

El proyecto sigue una arquitectura basada en microservicios con enfoque modular:



Tecnologías utilizadas

- Java 21
 - Quarkus 3.x
 - Maven
 - RESTEasy Reactive
 - Jackson (JSON)
 - JUnit 5 (testing)
 - Mockito (mocking)
-



Ejecución del proyecto



Requisitos

- JDK 17 o superior
- Maven 3.8+
- IntelliJ IDEA o VS Code



Modo desarrollo

```
./mvnw quarkus:dev
```



Compilar proyecto

```
./mvnw clean install
```



Ejecutar JAR

```
java -jar target/quarkus-app/quarkus-run.jar
```



Endpoints principales



Evaluar riesgo de cliente

POST /risk/evaluate

Request:

```
{
  "customerId": 123,
  "income": 2500,
  "debt": 800,
  "age": 30
}
```

Response:

```
{
  "riskLevel": "LOW",
  "approved": true,
  "score": 85
}
```

Lógica de negocio

El riesgo se calcula basado en:

- Nivel de ingresos

- Nivel de deuda
- Edad del cliente

Ejemplo de reglas:

- Si deuda > 70% ingreso → riesgo ALTO
- Si ingreso alto y deuda baja → riesgo BAJO
- Edad < 25 o > 65 → aumenta riesgo



Estructura de respuesta

Campo	Tipo	Descripción
riskLevel	String	LOW / MEDIUM / HIGH
approved	Boolean	Si el crédito es aprobado
score	Integer	Puntaje de riesgo (0-100)



Testing

Ejecutar pruebas unitarias:

```
./mvnw test
```



Errores comunes



Error: Maven no reconoce Quarkus plugin

✓ Solución:

- Verificar versión del plugin en `pom.xml`
- Ejecutar:

```
./mvnw clean
```



Error Java version

✓ Asegúrate de usar Java 17:

```
java -version
```

Buenas prácticas aplicadas

- Separación de capas (Controller / Service)
- DTOs para comunicación
- Validación de entrada
- Código desacoplado
- Uso de Quarkus dev mode

Mejoras futuras

- Integración con base de datos PostgreSQL
- Seguridad con JWT
- Dockerización del microservicio
- Integración con API Gateway
- Métricas con Prometheus

Si quieres, puedo ayudarte a lo siguiente:

- ☒ Arreglar el error exacto que te salía en Maven
- ☒ Revisar tu `pom.xml`
- ☒ Diseñar la arquitectura completa de tu prueba técnica
- ☒ O ayudarte a convertir esto en documentación tipo README para GitHub

Solo dime 